

MIA-aware FinFlex Cell Legalization with Power-Driven Cell Version Substitution

Da-Wei Huang and Shao-Yun Fang

Department of Electrical Engineering, National Taiwan University of Science and Technology, Taipei 106, Taiwan
{D11207008, syfang}@mail.ntust.edu.tw

Abstract—FinFlex standard cells have been proposed in sub-3 nm process nodes by TSMC to offer great design flexibility and a good trade-off between performance, power, and area efficiency. In the FinFlex technology, each cell type has a number of versions, varying in cell heights and threshold voltages, enabling flexible cell substitution for placement optimization. This paper presents the first work in the literature on standard cell legalization for the FinFlex technology, addressing cell version substitution, power efficiency, and minimum implant area (MIA) constraints. Our proposed algorithm comprises three stages: intra-row violation-aware pre-processing considering timing and power levels, directed acyclic graph-based initial legalization with cell version substitution, and dynamic programming-based inter-row violation removal. Experimental results show the superior performance of our approach over a baseline using state-of-the-art techniques.

I. INTRODUCTION

As reported in [15], TSMC adopts the FinFlex standard cell innovation in 3 nm process node to offer great design flexibility and good trade-off between performance, power, and area efficiency. For example, in the FinFlex 3-2 fin configuration, the cells placed on the rows equipped with 3 fins provide higher driving strengths compared to those placed on 2-fin rows. Different from the traditional 2-2 fin configuration, a 3-fin row has a larger row height compared to that of a 2-fin row, which is considered as non-integer multiple-cell-height or hybrid-row-height (HRH) designs in existing studies [1], [2], [5], [9], [12]. It is expected that HRH designs can also adopt multiple-row-height cells as those in conventional mixed-cell-height designs. For example, [2] and [12] reveal that a double-row-height cell spanning two rows can provide even larger driving strength than a taller but single-row-height cell. They consider four possible version variants of each standard cell considered. The version denoted as *SMALL* (S) of a cell represents its low-driven version, and *WIDE* (W), *TALL* (T), and *COMBO* (C) are the high-driven versions. Therefore, a *WIDE* cell is expected to be wider than a *SMALL* cell of the same cell type, and a *TALL* and a *COMBO* cell are thinner than a *WIDE* cell. *SMALL* and *WIDE* cells are placed in short rows, *TALL* cells in tall rows, and *COMBO* cells across a short row and a tall row.

The most critical challenge emerging from this novel design style is that it greatly increases the complexities of cell placement. Dobre et al. propose the first design optimization flow for HRH designs [5]. After that, Chen et al. propose an analytical hybrid-row-height-aware global placement algorithm and a dynamic block region generate approach [1]. Lin and Chang propose a cell legalization approach for HRH designs uniform or non-uniform row structures [9]. The three studies assume that cells have different cell heights (8T or 12T), but they do not consider multiple-row-height cells. In addition, as reported in [9], the legalization results suffer from 16% HPWL overhead compared to the initial global placement results, showing a huge gap in terms of wavelength optimization and the great design difficulty of such novel technology. To mitigate the problem, Liang et al. propose adopting cell version substitution during legalization [12], which allows a cell of the *SMALL* version to be switched to any of its high-driven versions and a high-driven cell can be switched to one of the other two high-driven versions, to guarantee timing and release the placement resource of short rows. The results in [12] show that applying cell version substitution can contribute to less than 1% wavelength overhead after legalization

This work was partially supported by TSMC, Synopsys, MPI, and NSTC of Taiwan under Grant No's NSTC 113-2927-I-011-502, 113-2640-E-002-001, 113-2640-E-006-001, 113-2222-E-011-005-MY3, and 114-2927-I-011-501.

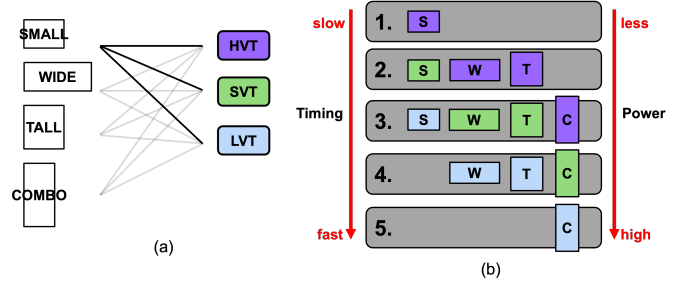


Fig. 1. The FinFlex technology offers multiple versions and threshold voltages (VTs) for each cell type. (a) Four cell sizes and three VTs. (b) Cell classification into five timing and power levels in this work.

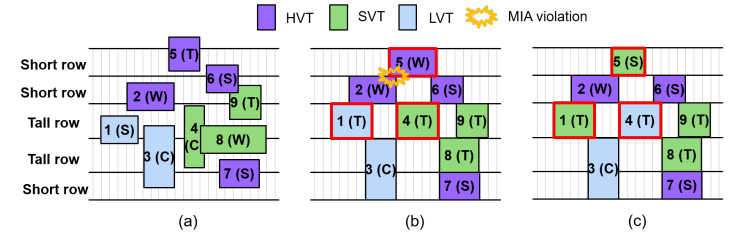


Fig. 2. Motivation of this work. (a) A legalization instance. (b) A legalization result with previous work. (c) An ideal result by considering VT swaps.

compared to the given global placement. However, the power overhead caused by transforming low-driven cells to high-driven versions is not considered. This insufficiency motivated this work in the first place.

It is important to note that in [15], multiple threshold voltages (VTs) are also offered in the FinFlex standard cell innovation in the 3 nm process node, and thus various combinations of the cell height and VT provide a huge optimization range in terms of performance and leakage power to meet different design purposes. For a single cell with a fixed height, adopting a lower VT can have a smaller delay while suffering from larger leakage power. When considering cell version substitution, adopting multiple VTs makes the operation more flexible and contributes to leakage power minimization. As illustrated in Fig. 1(a), if there are four cell sizes and three VT types, high VT (HVT), standard VT (SVT), and low VT (LVT), then the total number of available versions is 12. Each version exhibits unique timing and power characteristics, resulting in a more complex optimization scenario. To simplify the problem, we classify the 12 versions into five groups in this paper, which correspond to the five timing and power levels as shown in Fig. 1(b). The cells in the same group are assumed to have the same timing and power performance. To guarantee the timing and power closure of a design, a cell can only be substituted with a version belonging to the same group. For example, in power level 2, a *SMALL* cell with SVT can only be substituted with its *WIDE* or *TALL* version with HVT, according to our cell classification. This work can also effectively handle practical situations where the timing and power performances of all versions differ.

However, the designs using multiple threshold voltages must comply with an additional set of design rules due to the minimum implant area (MIA) constraint, which will be introduced in Section II-A. There have been many existing studies focusing on MIA-aware legalization and detailed placement problems [3], [4], [7], [8], [11], [13], [14], [16],

but they are not developed for HRH designs. Although dealing with the HRH legalization problem or handling the MIA-related rules alone is already difficult, we excitedly observed that cell version substitution in HRH designs may facilitate MIA compliance, which serves as the second motivation of this work. Figure 2(a) shows a legalization instance with the row configuration of HRH designs suggested by [2]. The cell colors indicate three VTs and cell versions are denoted by S, W, T, and C. Figure 2(b) shows a result the state-of-the-art work [12] may generate, where Cells 1, 4, 5, 8 are performed version substitution to minimize cell displacements, without considering VTs. Among them, Cell 5 experiences an MIA violation, Cell 1 causes power overhead, and Cell 4 suffers from timing degradation, according to our timing and power assumption in Fig. 1(b). As illustrated in Fig. 1(c), using different VTs for Cells 1 and 4 such that the new versions are in the same groups as the old ones can guarantee the timing and power performance. In addition, using a different version as well as a different VT for Cell 5 can even resolve the MIA violation.

This paper, therefore, proposes an algorithm flow for MIA-aware and power-driven HRH legalization with cell version substitution. The major contributions of this work are as follows:

- This is the first work in the literature to address MIA compliance in HRH designs through performance-driven cell version substitution.
- A timing and power level-aware cell clustering algorithm is proposed that leverages the flexibility of version substitution to eliminate potential intra-row MIA violations and contribute to better legalization results.
- A directed acyclic graph (DAG)-based legalization approach is developed to determine the best version of each cell contributing the minimized displacements.
- A dynamic programming-based method is adopted to systematically remove inter-row MIA violations.
- Experimental results demonstrate significant performance improvements in terms of cell displacement, MIA violations, and runtimes compared to the state-of-the-art approach [12].

In the following, Section II introduces some background knowledge of this work. Section III details the proposed algorithms. Finally, Sections IV and V respectively show the experimental results and briefly conclude this paper.

II. PRELIMINARIES

This section first explains design rules induced by the MIA constraint and then briefly introduces an optimal mixed-cell-height detailed placement algorithm, which inspires us to develop our dynamic programming-based MIA violation removal method.

A. Minimum Implant Area (MIA) Constraint

The MIA constraint sets a minimum implant area requirement, meaning a narrower cell can violate this rule if its neighboring cells have different threshold voltages, which results in an *intra-row MIA violation*, as the one shown in Fig. 3(a). It is common to transform the MIA constraint into a minimum width rule, min_w . In addition, two vertically adjacent cells with an overlap smaller than min_w are also prohibited and defined as an *inter-row MIA violation*, as the two vertically adjacent cells in Fig. 3(b). There are two common techniques to fix intra-row MIA violations. The first one is to directly insert filler cells beside a narrow cell to increase the implant width, as shown in Fig. 3(c). The second method is to cluster neighboring narrow cells with the same VT. Similarly, filler cell insertion (as illustrated in Fig. 3(d)) and cell shifting are usually used to resolve inter-row MIA violations. Considering that existing methods are not developed for HRH designs and typically address only a limited subset of the aforementioned operations, there is a strong need for effective algorithms that incorporate cell clustering, cell shifting, filler cell insertion, and cell version substitution.

B. Optimal Detailed Placement for Mixed-Cell-Height Designs

Huang et al. [6] propose a dynamic programming algorithm that can optimally solve the detailed placement problem for mixed-cell-height designs based on a given set of placement operations such as cell flipping and shifting. The algorithm breaks the traditional left-to-right cost propagation procedure in all previous studies and regards a growing group of multi-row-height cells as a sub-problem solved in each iteration. As shown in Fig. 4(a), [6] constructs a graph for the multi-row-height

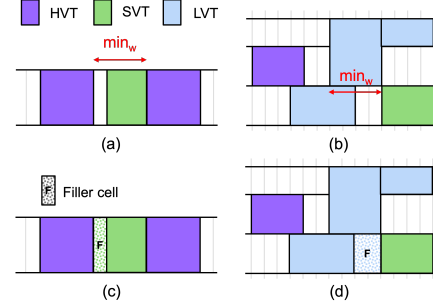


Fig. 3. MIA-induced design rule violations. (a) An intra-row violation. (b) An inter-row violation. (c)(d) Filler cell insertion for violation removal.

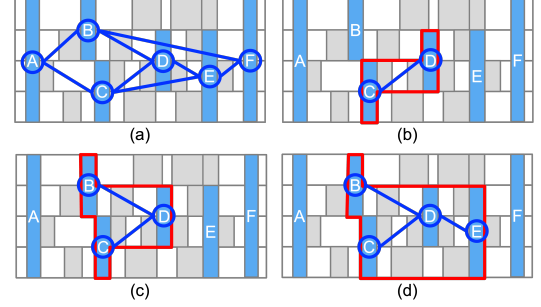


Fig. 4. The dynamic programming algorithm to derive optimal detailed placement for mixed-cell-height designs [6].

cells in a given placement, where each edge indicates that the two corresponding cells are adjacent by neglecting the single-row-height cells between them. In Fig. 4(b), the first subproblem may be composed of Cells C and D , deriving the optimal placement solution for each state combination of C and D . A state of a cell indicates a specific placement location and orientation for the cell. The second subproblem may be formed by clustering Cell B into the group, as shown in Fig. 4(c), enumerating the optimal solutions for each state combination of B , C , and D . Fig. 4(d) shows the third subproblem by incorporating E . Because after forming this subproblem, D is independent of the other multi-row-height cells outside the group, it can be omitted in the following process and its cost can be well considered in each state combination of B , C , and E . In this way, the optimal solution can be obtained when all multi-row-height cells are included in the group. Due to the optimality and the ability to consider the discrete states of each cell, we leverage this approach and make appropriate extensions in our work, which will be detailed below.

III. PROPOSED ALGORITHMS

This section presents the overall algorithm flow, followed by a detailed explanation of each step.

A. Overall Framework

Our algorithm for hybrid-row-height placement, incorporating cell version substitution and complex MIA constraints, is summarized in Fig. 5. It consists of three major stages: (1) the pre-processing stage, which addresses the intra-row MIA violation issue and determines cell ordering, (2) the legalization stage, which primarily determines the suitable location for each cell, selects its version, and resolves cell overlaps, and (3) the post-processing stage, which minimizes remaining inter-row violations while ensuring that previously resolved violations are not re-generated. To handle the intra-row MIA violations, we propose a filler-aware clustering algorithm. This method determines whether to cluster cells or insert filler cells for narrower cells based on the density of their surrounding areas, allowing for more effective utilization of empty space. To legalize cells, we propose a DAG-based approach that determines the version and position of cells simultaneously. This method offers cells maximum flexibility in placement while minimizing

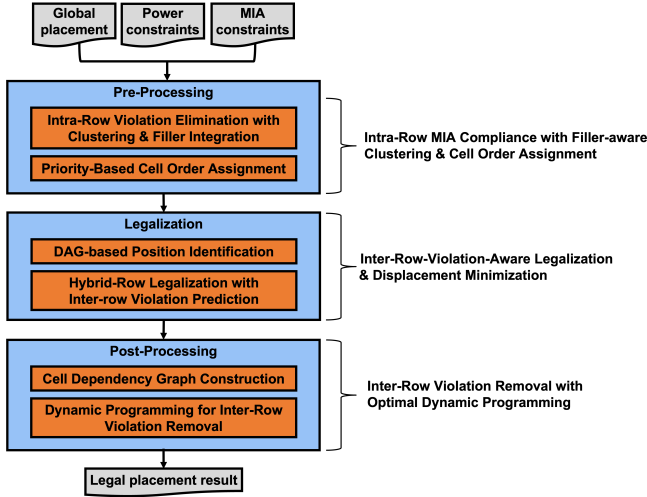


Fig. 5. The proposed algorithm framework.

displacement and avoiding inter-row MIA violations. Finally, we introduce new perspectives to mitigate inter-row violation. By simultaneously employing filler insertion and small amounts of cell shifting, great numbers of inter-row MIA violations can be removed. This method ensures that the subsequent detailed placement stage can address the remaining violations more easily and effectively.

B. Pre-Processing

In this section, we utilize cell cluster and filler cell insertion to resolve intra-row MIA violations and determine cell ordering for the following legalization step.

1) *Intra-Row Violation Elimination with Clustering & Filler Integration*: As introduced in Section II-A, an intra-row MIA violation is caused by a cell narrower than $\min_w$ and placed between two cells of different VTs. To resolve each potential intra-row violation, one technique involves adding a filler cell beside a narrow cell, and the other focuses on clustering cells with the same VT. The disadvantage of inserting filler cells lies in requiring additional space. On the other hand, clustering cells can lead to more cell displacements. Determining which cells should be assigned filler cells or clustered requires an effective strategy to minimize potential issues in the subsequent legalization stage.

To address this challenge, we propose a filler-aware clustering algorithm designed to effectively balance space utilization and displacement. For cells located in high-density regions, where available space is constrained, clustering can optimize space usage, freeing up more room for other cells and post-legalization optimization. Conversely, in lower-density regions, inserting filler cells can mitigate displacement while avoiding excessive resource consumption. To calculate the density of cells in a local region, we establish a window centered at each cell, extending outward by a distance equal to the combined height of a short row and a tall row. The total cell area within this window is then divided by the effective area of the window to obtain the density. To accelerate the density calculation, we utilize a QuadTree structure to record cell positions, achieving a time complexity of $\mathcal{O}(\log n)$.

A clustering graph $G(V, E, W)$ is constructed to facilitate the cell clustering task, as shown in Fig. 6(a). Each cell narrower than $\min_w$ is modeled as a vertex v_i , and it is connected to other vertices representing the other cells in the window centering at v_i with the same power and timing level. Each edge indicates that the two vertices form a clustering candidate. Note that although clustering two cells with the same VT is more intuitive to directly resolve a potential intra-row violation for a narrow cell (which is also a common way adopted in previous studies), we found that clustering two cells of the same power and timing level contributes to larger flexibility during the legalization stage. Fig. 6(b) demonstrates an example of clustering two cells belonging to the third power and timing level. With cell version substitution, four possibilities are present to cluster them, and the option most favorable for legalization can be selected. In contrast, Fig. 6(c) shows an example of clustering two cells with the same VT between in different timing and power levels.

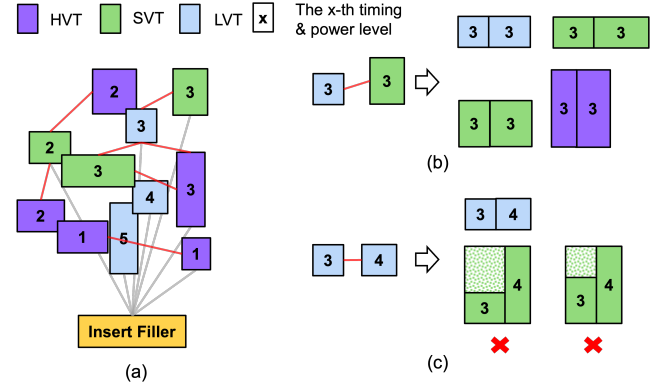


Fig. 6. Cell clustering to eliminate potential intra-row MIA violations. (a) A clustering graph. (b) Clustering two cells with the same timing and power level. (c) Clustering two cells with different levels.

Algorithm 1: Filler-aware clustering

Input: A global placement result
Output: Intra-row MIA free result

```

1 Construct the clustering graph  $G(V, E, W)$ ;
2 repeat
3   Find an edge  $e_{uv} \in E$  with the minimum weight  $w_{uv}$ ;
4   if  $e_{uv}$  is connected to the filler vertex then
5     Set  $x$  as the non-filler vertex between  $u$  and  $v$ ;
6      $x.Is\_Filler \leftarrow TRUE$  and  $x.Is\_Cluster \leftarrow FALSE$ ;
7     Delete  $e_{uv}$ ;
8   end
9   else
10     $u.Is\_Filler \leftarrow FALSE$  and  $u.Is\_Cluster \leftarrow TRUE$ ;
11     $v.Is\_Filler \leftarrow FALSE$  and  $v.Is\_Cluster \leftarrow TRUE$ ;
12    if both  $u$  and  $v$  are cell vertices then
13      Group  $u$  and  $v$  as a cluster  $u_\ell$ ;
14      Set the position of  $u_\ell$  at the median point of  $u$  and  $v$ ;
15      Assign the density of  $u_\ell$  as the maximum density of  $u$  and  $v$ ;
16      Assign the width of  $u_\ell$  as the total width of  $u$  and  $v$ ;
17      Delete the edge  $e_{uv}$ ;
18      Delete the edge  $e_{ui} \in E$ , if Cell  $i$  is wider than  $\min_w$ ;
19      Delete the edge  $e_{vj} \in E$ , if Cell  $j$  is wider than  $\min_w$ ;
20      Update the edge weights connecting to  $u_\ell$  by Equation (2);
21    end
22  end
23  else
24    Set  $x$  as the cell vertex and set  $u_\ell$  as the cluster;
25    Merge cell  $x$  into  $u_\ell$ ;
26    Update the position, width, and density of  $u_\ell$ ;
27    Delete the edge  $e_{xu_\ell}$ ;
28    Delete the edge  $e_{xi} \in E$  if Cell  $i$  is wider than  $\min_w$ ;
29    Update the edges connecting to  $u_\ell$  by Equation (2);
30  end
31 until  $E == \emptyset$ ;
32 return;
```

Since a cell can only be substituted with a version in the same level, only one feasible combination exists. Although we may also switch the VTs of the two cells such that two more combinations can be considered, filler cell insertion is then required, ultimately offering no advantage to the cell clustering operation.

In addition to cell vertices, a vertex representing filler cell insertion is also constructed in the clustering graph and connected to the vertices of narrow cells, indicating that each narrow cell can also be clustered with a filler cell. Each edge is associated with an edge weight $w_{uv} \in W$ calculated as follows:

$$w_{uv} = \theta_{uv} \times (\alpha_1 \times dist_{uv} + \alpha_2 \times width_{uv} - \alpha_3 \times density_{uv}), \quad (1)$$

where $dist_{uv}$ is the Manhattan distance between Cells u and v , $width_{uv}$ is the total cell width of u and v , and $density_{uv}$ is the larger local density of the two windows of the two cells. α_1 , α_2 , and α_3 are user-

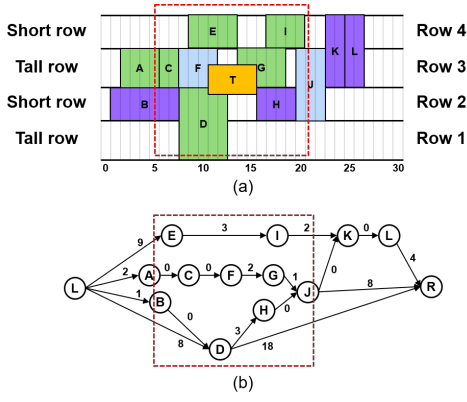


Fig. 7. DAG-based cell legalization. (a) A target cell T being legalized and other already legalized cells. (b) The DAG and the region to find candidate legalization locations for T .

defined parameters used for normalization such that the three terms fall in the range of $[0, 1]$. Note that if a vertex of the edge e_{uv} is the filler vertex, $dist_{uv}$ is set as one-quarter of the window width, discouraging cell clustering when the distance of the two cells exceeds this value. θ_{uv} is set as 0.5 if both the two cells are narrower than min_w ; otherwise, θ is set as 1.

With the clustering graph G , the proposed filler-aware clustering method is outlined in Algorithm 1. The algorithm iteratively clusters two cells or a cell and a cluster with the minimum edge weight in each iteration. If one of the vertices of an edge is the filler vertex, the two flags Is_Filler and $Is_Cluster$ are marked accordingly (Lines 4–8). If both the two vertices represent cells, we merge the two cells as a cluster u_ℓ , compute the position, density, and width for u_ℓ , and delete edges that are not necessary for the following (Lines 12–21). A similar process is applied for the case when one of the two vertices is already a cluster (Lines 22–29).

2) *Priority-Based Cell Order Assignment*: An effective cell ordering strategy can lead to faster convergence during sequential legalization and help minimize displacement as well. In our approach, cells that are wider, located in dense areas, or have fewer available versions are given higher priorities. In contrast, cells that do not meet these criteria have more placement options, making them easier to legalize. This prioritization ensures that critical cells are addressed first, facilitating a smoother and more efficient legalization process. Therefore, the priority of each cell i is defined as follows:

$$P_i = \beta_1 \times width_i + \beta_2 \times density_i + \beta_3 \times \frac{1}{candidate_i}, \quad (2)$$

where $width_i$, $density_i$, and $candidate_i$ are the width, density, and the number of candidate versions with the same timing and power level of Cell i . A cell with a higher P_i has a priority to be legalized.

C. Legalization

After the pre-processing stage, all potential intra-row violations are eliminated. In this stage, the final version of each cell is determined and is placed on a legal row to minimize displacement and the possibility of incurring inter-row violations. This step is crucial for ensuring outstanding placement performance with minimal inter-row violations. To legalize cells efficiently, We adopt a directed acyclic graph (DAG) as a horizontal constraint graph G to record the current legalization result, as shown in Fig. 7. A directed edge is connected from cell u to v in G if v is immediately on the right to u in some rows. The left and right boundaries and macros are treated as fixed cells. Each edge weight indicates the distance between the two legalized cells.

1) *DAG-based Position Identification*: The legalization procedure is performed sequentially. For a target cell T being legalized, we use the window of T to identify its candidate intervals in the DAG. As illustrated in Fig. 7(b), all edges within the window are considered potential placement candidates for T . For a non-COMBO version, the process involves straightforwardly selecting the appropriate candidate row for each candidate edge. In contrast, for a COMBO version, the

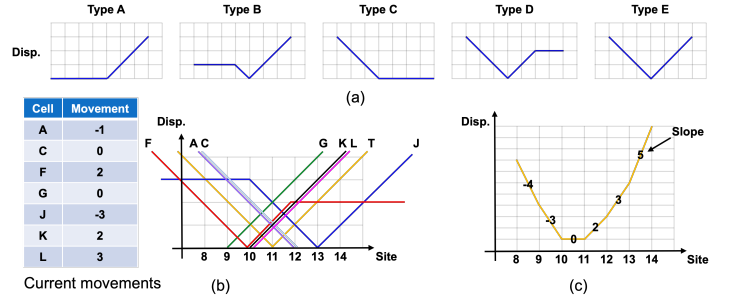


Fig. 8. Total displacement calculation for a candidate interval. (a) The five displacement curves [10]. (b) The displacement curves of the affected cells if T is inserted in the interval (FG). (c) The final displacement curve by merging those in (b).

approach combines candidate intervals across adjacent rows, including both short and tall rows. To enhance the quality of the placement result, once the *cluster* is placed, it is de-clustered back to its original configuration. This de-clustering allows for more space to be utilized when searching for candidate intervals in subsequent steps, improving overall placement flexibility and efficiency. To prevent intra-row violations from reoccurring, when selecting the intervals between de-clustered cells, it is essential to ensure that the VT of the target cell matches that of the de-clustered cells.

As illustrated in Fig. 7(b), for the target cell T of a non-COMBO version, the candidate intervals for Cell T are (LD, DR) , (BD, DH, HJ) , (AC, CF, FG, GJ) , (LE, EI, IK) , separately for the four rows from bottom to top. If T is a COMBO version, the candidate intervals between Rows 1 and 2 are the combinations of $\{(LD, DR), (BD, DH, HJ)\}$. Notice that when enumerating the combinations of two adjacent rows, we need to ensure their legality. For example, the candidate interval (LD, DH) and (LD, HJ) on Row 1 and 2 are illegal combinations, because T cannot simultaneously be placed on the right and left sides of Cell D . Moreover, since candidate (AC) is a *cluster*, the target cell must have the same VT when selecting this interval to prevent the intra-row violation on Cell C .

When evaluating the displacement of inserting T at each candidate interval, we need to consider the *movements* of other legalized cells. The *movement* of a cell represents the changes in the x-coordinate of a cell from its global position to its current position. When the *movement* is positive/negative, it indicates that the cell is positioned on the right/left of its global position, respectively. To evaluate the change in overall displacement after placing the target cell in a candidate interval, we use the five displacement curves proposed in [10], as illustrated in Fig. 8. For the placed cells on the right side of the target cell, the resulting curve will be classified as either type A or B. On the other hand, if the placed cells are on the left side of the target cell, the resulting curve will be classified as either type C or D. Type E represents the displacement curve specifically for the target cell.

When the target cell T is placed into a candidate interval, it may cause overlap with neighboring cells on both sides, making it essential to assess the extent of the impact. First, we calculate the total overlap created by inserting T and identify the non-overlapping sites with the adjacent cells. As illustrated in Fig. 7(a), for the interval (FG) , T needs to be placed at *site* 12 to be non-overlapped with F , and it needs to be placed at *site* 9 to be non-overlapped with G , with a total overlap of 3. Referring to Fig. 8(b), and based on the left table, the current *movement* of Cell F is 2, and thus its displacement curve is classified as type D (shown in red). This indicates that when Cell T is placed right to *site* 12, Cell F does not need any shifting, resulting in a horizontal curve segment. When Cell T is placed between *site* 10 to 12, the displacement decreases, because Cell F moves toward its global position. When Cell T is placed left to *site* 10, the displacement increases. On the other side, based on the left table, since the current *movement* of Cell G is 0, its displacement curve is type A (shown in green). This means that when Cell T is placed right to *site* 9, the displacement of Cell G increases linearly. The turning points in the displacement curve for Cell F are at sites 10 and 12, while the turning point for Cell G is at site 9.

Afterward, we need to address the overlapping issue by shifting the

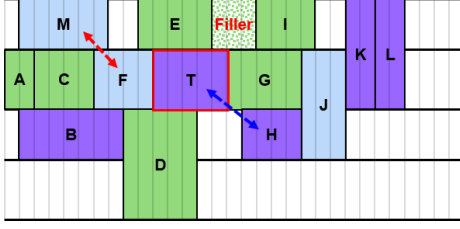


Fig. 9. Inter-row MIA violation prediction during legalization.

cells outward. We use the DAG structure and Breadth-First Search (BFS) to find out whether sufficient space exists to eliminate the overlap. This approach can accurately calculate the total displacement of affected cells, which is the major difference between our approach and that in [10]. BFS traverses all affected cells on both sides. As it encounters available space, the overlap is progressively reduced until it becomes less than or equal to 0. Continuing the previous example in Fig. 7(a), since the overlap is 3, we first try to resolve the overlap by traversing to the right side. Upon reaching the next cell, *J*, since there is 1-site spacing between *G* and *J*, the overlap is reduced to 2. However, since overlap still exists, the traversal must proceed further to the right to fully resolve it. In the lower row of *J*, the overlap can be resolved due to the 8 available sites between *J* and the right boundary. In the upper row of *J*, the traversal encounters *K* and *L* sequentially. With no spacing between them, both *K* and *L* are affected by Cell *T*, and the traversal continues to the right until the overlap is completely eliminated. For each affected cell, a displacement curve is constructed based on its current *movement* and using the similar procedure mentioned above, as shown in Fig. 8(b).

After traversing both sides, the displacement curves of all affected cells are merged into a single curve, as shown in Fig. 8(c). The resulting total displacement curve forms a convex function, indicating that shifting the target cell will either maintain or increase the overall displacement. Constructing the total displacement curve involves summing the slopes at each turning point from left to right. Following this, we identify the segments where the slope changed from negative to non-negative or from non-negative to positive, which represent the best intervals for placing the target cell. In the example shown in Fig. 8(c), the target cell can be placed at site 10 or 11, resulting in minimized displacement with this ordering.

Note that we assign a large cost if BFS traverses to a macro, preventing the macro from being moved. In addition, in some dense regions, placing the target cell in a candidate interval located in a dense region may move cells outward very far, which is time-consuming and undesirable. Thus, we set an upper bound for BFS traversal to avoid such non-ideal situations.

2) Hybrid-Row Legalization with Inter-Row Violation Prediction: In addition to using the total displacement of affected cells as the metric to evaluate the quality of placing the target cell in an interval, considering whether it has a high possibility of resulting in inter-row MIA violations is also important. In previous works, inter-row violations were typically checked for all cells in adjacent rows, which is time-consuming and can lead to pessimistic results. In fact, if multiple cells with the same VT are placed in the same row consecutively, the white spaces among them can be filled with filler cells of the same VT. Therefore, it is only necessary to check the leftmost and the rightmost cells for inter-row violations. As shown in Fig. 9, an inter-row violation can be asserted by only checking Cells *G* and *I*. However, since Cells *E* and *I* have the same VT, filler cells with the same VT can be inserted between them, thereby eliminating any inter-row violation between *G* and *I*. This ensures that the estimation will not result in errors when checking Cell *G* and *I* due to the presence of the filler cell.

When temporarily placing the target cell in a candidate interval and shifting all affected cells without overlaps, we will check whether the placement may result in a significant number of inter-row violations. Additionally, we also check potential violations within a user-defined distance to ensure comprehensive validation. If the placement result generates numerous violations and potential violations, a penalty is assigned to the candidate based on the severity of these violations. As shown in Fig. 9, when Cell *T* is temporarily placed in the interval (*FG*) and given a user-defined distance of two sites, the result shows one

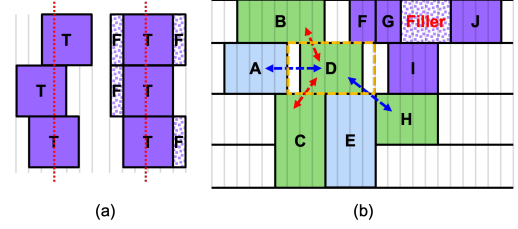


Fig. 10. Dynamic programming-based inter-row violation removal. (a) Six available states of each cell for a specific cell shifting and filler insertion scenario. (b) A placement instances with two inter-row violations and two potential violations.

violation and one potential violation, respectively indicated with red and blue double arrows. Note that we only perform this violation prediction procedure if the target cell is located in a dense region, because inter-row MIA violations can be easily removed in sparse regions.

D. Post-Processing

When all cells are legalized, inter-row violations may still occur in the placement. To mitigate violations, we extend the Dynamic Programming (DP) algorithm introduced in Section II-B to systematically remove these violations. In addition to utilizing cell shifting as in all existing studies, we also consider filler cell insertion, which is an effective inter-row violation removal method without cell displacements for sparse regions.

1) Cell Dependency Graph Construction: The dynamic programming algorithm introduced in Section II-B can theoretically generate an optimal placement refinement result under a pre-defined set of available cell states. To resolve inter-row violations, we apply cell shifting with a pre-defined maximum cell shifting and a pre-defined set of filler cell insertion configurations. As illustrated in Fig. 10(a), if the maximum cell shifting is limited to one placement site and we only consider inserting a single unit filler cell to the left or right adjacent to a target cell, there are overall six states for each cell.

Similar to [6], we construct a graph for a set of vertices representing cells that are dependent on each other considering cell overlaps and inter-row violations. Different from that in [6], cell dependency is not restricted to horizontally adjacent cells; it will also occur between vertically adjacent cells. As shown in Fig. 10(b), Cell *D* is dependent on Cells *B* and *C* due to the current inter-row violations. In addition, according to the pre-defined maximum cell shifting of one placement site, the yellow box indicates the possible location where *D* is placed after shifting. Therefore, *D* is also dependent on Cells *A* and *H*, because *D* and *A* may overlap, and *D* and *H* may suffer from an inter-row violation, after shifting. As a result, in the graph constructed for the instance in Fig. 10(b), *D* connects to the other four cells highlighted by the double arrows. The vertex cost for each cell is determined by the cell displacement from its global position represented by the corresponding state. Each edge is associated with a cost table, recording the penalties due to cell overlap and inter-row violations caused by each state combination of the two connected cells. It is important to note that the costs induced by cell overlaps must be set higher than the cost of inter-row violations to prioritize preventing overlaps.

2) Dynamic Programming for Inter-Row Violation Removal: Constructing the dependency graph of a given placement, the dynamic programming-based algorithm introduced in Section II-B is applied to find the optimal refined placement without cell overlaps and with the minimized number of inter-row violations. Since we simultaneously consider cell shifting and filler cell insertion by enumerating cell states, the total cell displacement can also be optimized by inserting filler cells in sparse regions and automatically determining which side a filler cell will be inserted by a target cell.

Note that the cells in a dense region could be very compact, increasing the likelihood of overlaps and inter-row violations when cells are shifted. The dependency graph constructed for a dense region may be very complex with many edges, and solving the graph during the dynamic programming process can lead to a much longer runtime or even memory explosion. Therefore, we set an upper bound on the number of vertices considered at a time for dense regions. In this way, the placement

TABLE I
EXPERIMENTAL RESULTS ON LEGALIZATION PERFORMANCE.

Benchmark		Enhanced [12]			Ours		
Case	Init_Density (%)	Total Disp.	Density (%)	CPU (s)	Total Disp.	Density (%)	CPU (s)
des_perf_a_md1	59.27	248,819	59.22	64.536	102,829	56.15	46.723
des_perf_a_md2	58.75	185,649	58.18	58.014	95,200	54.65	34.122
des_perf_b_md1	70.16	128,399	67.69	27.708	74,003	61.85	2.391
des_perf_b_md2	72.19	128,985	69.89	39.166	73,159	64.20	3.260
edit_dist_l_md1	69.04	168,358	64.75	9.055	82,714	60.21	2.438
edit_dist_a_md2	66.90	170,187	63.54	32.946	95,166	59.26	3.341
edit_dist_a_md3	63.38	227,345	60.96	47.833	115,694	57.26	19.436
fft_2_md2	76.03	40,705	74.27	10.774	23,164	71.18	0.726
fft_a_md2	37.05	32,642	36.03	0.349	15,341	32.63	0.293
fft_a_md3	37.10	33,076	35.26	0.338	15,385	32.21	0.298
pci_bridge32_a_md1	54.40	35,279	54.31	5.859	25,806	48.80	0.627
pci_bridge32_a_md2	55.57	37,475	55.28	12.156	21,522	49.88	4.120
pci_bridge32_b_md1	26.01	35,967	25.62	1.282	16,908	22.86	0.328
pci_bridge32_b_md2	21.14	36,155	21.02	4.648	18,479	18.88	0.397
pci_bridge32_b_md3	22.99	37,243	22.37	4.613	19,031	19.99	0.434
Normalized	-	1.000	1.000	1.000	0.473	0.895	0.622

TABLE II
EXPERIMENTAL RESULTS ON INTER-ROW VIOLATIONS.

Benchmark	Enhanced [12]		Ours			
			w/o Prediction		w/ Prediction	
Case	I_IRV.	F_IRV.	I_IRV.	F_IRV.	I_IRV.	F_IRV.
des_perf_a_md1	7,540	4,467	9,310	3,849	8,311	2,859
des_perf_a_md2	7,364	4,033	9,276	2,604	8,294	1,789
des_perf_b_md1	7,473	1,606	9,440	356	8,268	186
des_perf_b_md2	7,902	2,377	9,818	507	9,024	311
edit_dist_l_md1	5,778	497	7,888	131	7,454	83
edit_dist_a_md2	6,606	1,959	8,588	463	7,445	198
edit_dist_a_md3	6,843	2,796	8,780	1,801	7,478	1,137
fft_2_md2	2,114	577	2,373	104	2,128	58
fft_a_md2	1,023	16	1,348	9	1,287	3
fft_a_md3	1,037	9	1,470	9	1,419	2
pci_bridge32_a_md1	1,985	454	2,306	120	2,139	69
pci_bridge32_a_md2	2,324	872	2,464	372	2,278	223
pci_bridge32_b_md1	1,385	106	1,648	21	1,584	11
pci_bridge32_b_md2	1,515	247	1,889	60	1,826	37
pci_bridge32_b_md3	1,592	299	1,933	48	1,805	43
Normalized	1.000	0.325	-	-	-	-
	-	-	1.000	0.093	0.918	0.061

refinement procedure can be conducted in a much efficient way with such a divide-and-conquer strategy.

IV. EXPERIMENTAL RESULTS

We implemented our MIA-aware FinFlex Cell legalization with C++ programming language and conducted the experiments on a 3.9 GHz Linux machine with 62 GB of memory. We adopted and modified the benchmark circuits from the ICCAD-2017 CAD Contest in Multi-Deck Standard-Cell Legalization [17]. We ignored the fence region constraint and kept all macros in the same positions. To preserve the original placement area, we assume that the heights of a short row and a tall row are respectively 0.8 and 1.2 times the height of the original row. We adopt the following procedure to generate cells conforming to the cell versions considered in this work. For each cell in the given global placement result of a benchmark circuit, we first ignore its original cell height, and its original width w is regarded as the cell width of its SMALL version. Then the cell is randomly assigned to a timing and power level, along with a randomly selected version and threshold voltage within that level. If the cell is selected as a TALL/WIDE/COMBO version, its width is set as $1w/2w/0.7w$. The maximum cell shifting is set as one placement site and the three filler cell insertion configurations mentioned in Section III-D1 are adopted, and thus the six states in Fig. 10(a) are used in our implementation for dynamic programming-based inter-row violation removal.

To demonstrate the effectiveness of the proposed algorithm flow, we compare our results with [12], which is the state-of-the-art legalization work for HRH design. [12] separates cell version substitution and legalization into two distinct stages, and the version substitution stage only ensures that the densities of short and tall cells of each

local region are below a maximum threshold. Consequently, during the legalization stage, cell versions cannot be further adjusted based on the current placement conditions. Since [12] only determines cell versions and legalizes cells without considering the MIA constraint, for a fair comparison, we integrate the cell clustering technique proposed in [3] and our dynamic programming-based post-processing approach to enhance [12] with MIA awareness.

Table I shows the legalization performances of [12] and our approach. The column “Init_Density” gives the initial density of each benchmark, and the columns “Total Disp.”, “Density”, and “CPU” separately represent the total cell displacement in terms of sites, the final placement density, and the runtime in seconds. The results show that our algorithm averagely achieves a 53% improvement in the total cell displacement, a 10% reduction in placement density, and a 34% reduction in runtime. The enhanced performance in placement is due to the simultaneous consideration of cell version substitution and filler cell insertion when performing cell clustering. In addition, the flexibility of our approach allowing cell version substitution in the legalization stage further contributes to minimizing displacement.

Table II presents the results on inter-row violations at different stages, comparing the outcomes with and without using the proposed predicted model. The column “I_IRV” represents the number of inter-row violations after the legalization stage, and “F_IRV” indicates that after dynamic programming-based post-processing optimization. “w/ Prediction” and “w/o Prediction” respectively indicate the results with and without the use of the inter-row violation prediction approach during legalization. Several features can be found in the results. First, our results have larger numbers of initial inter-row MIA violations, which is because the proposed legalization approach generates more compact placements to minimize the total displacement. Second, the proposed dynamic programming-based inter-row violation removal approach is very effective and efficient. With slight shifts and three simple filler cell insertion configurations applied through the dynamic programming-based post-processing approach, the number of violations decreases significantly for every approach, resulting in at most 94% violation removal. Third, utilizing the predicted model not only reduces the inter-row violations after the legalization stage but also results in lower numbers of remaining violations. Since the approach of [12] can result in many high-density local regions and cells are placed without considering MIA violations, the violations in the dense regions are difficult to be removed in the post-processing stage with minor cell adjustments.

V. CONCLUSIONS

This paper introduces a novel approach for standard cell legalization in the TSMC’s FinFlex technology for sub-3 nm nodes, focusing on cell version substitution, power and timing efficiency, and MIA-induced design rules. The proposed three-stage method composed of pre-processing, DAG-based legalization, and dynamic programming for inter-row violation removal optimizes cell placement and minimizes violations. Experimental results demonstrate that this approach outperforms existing state-of-the-art techniques, enhancing design flexibility, and offering an efficient solution for FinFlex-based IC design.

REFERENCES

- [1] J. Chen, Z. Huang, Y. Huang, W. Zhu, J. Yu, and Y.-W. Chang, "An Efficient EPIS Algorithm for Global Placement with Non-Integer Multiple-Height Cells," in *proceedings of Design Automation Conference (DAC)*, 2020.
- [2] K.-Y. Chen, H.-C. Hsu, W.-K. Mak, and T.-C. Wang, "HybridGP: Global Placement for Hybrid-Row-Height Designs," in *proceedings of Asia South Pacific Design Automation Conference (ASP-DAC)*, 2022.
- [3] J. Chen, P. Yang, X. Li, W. Zhu, and Y.-W. Chang, "Mixed-Cell-Height Placement with Complex Minimum-Implant-Area Constraints," in *proceedings of International Conference on Computer-Aided Design (ICCAD)*, 2018.
- [4] J. Chen, Y.-W. Chang, and Y.-Y. Wu, "Mixed-Cell-Height Detailed Placement Considering Complex Minimum-Implant-Area Constraints," *Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, vol. 40, no. 10, pp. 2128–2141, 2021.
- [5] S. A. Dobre, A. B. Kahng, and J. Li, "Design Implementation With Noninteger Multiple-Height Cells for Improved Design Quality in Advanced Nodes," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 37, no. 4, 2018.
- [6] D.-W. Huang, Y.-J. Jiang, and S.-Y. Fang, "Spacing Cost-aware Optimal and Efficient Mixed-Cell-Height Detailed Placement for DFM Considerations," *proceedings of International Conference on Computer-Aided Design (ICCAD)*, 2023.
- [7] E. Jeong, H. Park, and T. Kim, "A Systematic Removal of Minimum Implant Area Violations under Timing Constraint," in *proceedings of Design, Automation & Test in Europe (DATE)*, 2022.
- [8] A. B. Kahng and H. Lee, "Minimum Implant Area-Aware Gate Sizing and Placement," in *proceedings of Great Lakes Symposium on VLSI (GLSVLSI)*, 2014.
- [9] Z.-Y. Lin and Y.-W. Chang, "A Row-Based Algorithm for Non-Integer Multiple-Cell-Height Placement," in *proceedings of International Conference on Computer-Aided Design (ICCAD)*, 2021.
- [10] H. Li, W.-K. Chow, G. Chen, B. Yu, E. F.Y. Young, "Pin-Accessible Legalization for Mixed-Cell-Height Circuits," *Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)*, vol. 41, no. 1, pp. 143–154, 2022.
- [11] H.-C. Lin and S.-Y. Fang, "MIA-aware Detailed Placement and VT Reassignment for Leakage Power Optimization," in *proceedings of Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2023.
- [12] S.-H. Liang, T.-L. Hsiung, W.-K. Mak, and T.-C. Wang, "Hybrid-Row-Height Design Placement Legalization Considering Cell Variants," in *proceedings of ACM Great Lakes Symposium on VLSI (GLSVLSI)*, 2023.
- [13] S.-I. Lei, W.-K. Mak, and C. Chu, "Minimum Implant Area-Aware Placement and Threshold Voltage Refinement," in *proceedings of Asia South Pacific Design Automation Conference (ASP-DAC)*, 2016.
- [14] K.-H. Tseng, Y.-W. Chang, and C. C.-C. Liu, "Minimum-Implant-Area-Aware Detailed Placement with Spacing Constraints," in *proceedings of ACM/IEEE Design Automation Conference (DAC)*, 2016.
- [15] S.-Y. Wu, C.-H. Chang, M.-C. Chiang, C.-Y. Lin, J.-J. Liaw, J.-Y. Cheng, J.-Y. Yeh, H.-F. Chen, S.-Y. Chang, K.-T. Lai, M.-S. Liang, K.-H. Pan, J.-H. Chen, V.-S. Chang, T.-C. Luo, X. Wang, Y.-S. Mor, C.-I. Lin, S.-H. Wang, M.-Y. Hsieh, C.-Y. Chen, B.-F. Wu, C.-J. Lin, C.-S. Liang, C.-P. Tsao, C.-T. Li, C.-H. Chen, C.-H. Hsieh, H.-H. Liu, P.-N. Chen, C.-C. Chen, R. Chen, Y.-C. Yeo, C.-O. Chui, W. Chang, T.-L. Lee, K.-B. Huang, H.-J. Lin, K.-W. Chen, M.-H. Tsai, K.-S. Chen, X.-M. Chen, Y.-K. Cheng, C.-H. Wang, W. Shue, Y.-Ku, S.-M. Jang, M. Cao, L.-C. Lu, and T.-S. Chang, "A 3nm CMOS FinFlex™ Platform Technology with Enhanced Power Efficiency and Performance for Mobile SoC and High Performance Computing Applications," in *proceedings of International Electron Devices Meeting (IEDM)*, 2022.
- [16] Z. Zhu, Z. Huang, P. Yang, W. Zhu, J. Chen, H. Zhou, and S. Dong, "Mixed-Cell-Height Legalization Considering Complex Minimum Width Constraints and Half-Row Fragmentation Effect," *Integration*, vol. 71, no. C, pp. 1–10, 2020.
- [17] ICCAD-2017 CAD Contest in Multi-Deck Standard-Cell Legalization, <http://iccad-contest.org/2017/index.html>